UNIVERSIDADE DA AMAZÔNIA – UNAMA
[NOME DO CURSO]

[NOME COMPLETO DO INTEGRANTE 1]
[NOME COMPLETO DO INTEGRANTE 2]

DEVOPS NA PRÁTICA:

pipeline de CI/CD com GitHub Actions, orquestração com Kubernetes e Helm e monitoramento com Elastic Stack

BELÉM – PA
2026

[NOME COMPLETO DO INTEGRANTE 1]
[NOME COMPLETO DO INTEGRANTE 2]

DEVOPS NA PRÁTICA:

pipeline de CI/CD com GitHub Actions, orquestração com Kubernetes e Helm e monitoramento com Elastic Stack

Relatório técnico apresentado à disciplina DevOps Tools, do curso de [Nome do Curso] da Universidade da Amazônia (UNAMA), como requisito parcial de avaliação da Atividade Avaliativa – DevOps na Prática.

Professor: Prof. Rodrigo Silva Marques.

BELÉM – PA
2026

RESUMO

Este relatório descreve a construção de um projeto prático de DevOps que integra conceitos, ferramentas e práticas estudados na disciplina DevOps Tools. Inicialmente, apresentam-se os fundamentos de DevOps e sua relação com as metodologias ágeis, as diferenças entre Integração Contínua, Entrega Contínua e Implantação Contínua, e a comparação entre máquinas virtuais e containers. Em seguida, detalha-se a implementação de uma API de tarefas em Node.js com testes automatizados, empacotada em imagem Docker e submetida a um pipeline no GitHub Actions que executa os testes, constrói a imagem, publica-a no GitHub Container Registry e realiza a implantação automática em um cluster Kubernetes por meio de um chart Helm, expondo a aplicação via serviço NodePort. Para a observabilidade, configurou-se a Elastic Stack (Filebeat, Elasticsearch e Kibana) em containers, com logs estruturados no padrão Elastic Common Schema e um dashboard que exibe os logs em tempo real, a distribuição de códigos de status HTTP e o tempo de resposta das requisições. Por fim, relatam-se os desafios enfrentados e apresenta-se uma reflexão crítica sobre como DevOps agrega valor ao ciclo de vida do software, concluindo-se que a automação ponta a ponta reduz o tempo entre a escrita do código e sua disponibilização, aumenta a confiabilidade das entregas e fornece retorno rápido sobre o comportamento da aplicação em execução.

Palavras-chave: DevOps. Integração contínua. Kubernetes. Helm. Elastic Stack.

ABSTRACT

This report describes a hands-on DevOps project that integrates the concepts, tools and practices studied in the DevOps Tools course. It first presents the fundamentals of DevOps and its relationship with agile methodologies, the differences between Continuous Integration, Continuous Delivery and Continuous Deployment, and a comparison between virtual machines and containers. It then details the implementation of a Node.js task API with automated tests, packaged as a Docker image and processed by a GitHub Actions pipeline that runs the tests, builds the image, pushes it to the GitHub Container Registry and automatically deploys it to a Kubernetes cluster through a Helm chart, exposing the application through a NodePort service. For observability, the Elastic Stack (Filebeat, Elasticsearch and Kibana) was configured in containers, with structured logs following the Elastic Common Schema and a dashboard showing real-time logs, HTTP status code distribution and request response times. Finally, the challenges faced are reported and a critical reflection on how DevOps adds value to the software life cycle is presented.

Keywords: DevOps. Continuous integration. Kubernetes. Helm. Elastic Stack.

LISTA DE FIGURAS

Figura 1 – Arquitetura da solução: CI/CD, orquestração e observabilidade.....16
Figura 2 – Execução dos testes automatizados com cobertura de código.....19
Figura 3 – Pipeline CI/CD executado com sucesso no GitHub Actions.....20
Figura 4 – Serviço da aplicação rodando no Kubernetes (Minikube) e exposto via NodePort 22
Figura 5 – Validação do chart (helm lint) e teste do release (helm test).....22
Figura 6 – Aplicação acessada pelo navegador por meio do NodePort 30080.....22
Figura 7 – Coleta dos logs pelo Filebeat e indexação no Elasticsearch.....24
Figura 8 – Dashboard configurado no Kibana com logs em tempo real, status code e tempo de resposta.....25

LISTA DE QUADROS

Quadro 1 – Etapas da atividade e onde cada entrega é atendida.....9
Quadro 2 – Relação entre metodologias ágeis e DevOps.....11
Quadro 3 – Comparação entre CI, Continuous Delivery e Continuous Deployment.....12
Quadro 4 – Comparação entre máquinas virtuais e containers.....13
Quadro 5 – Principais Beats da Elastic Stack.....15
Quadro 6 – Ferramentas e versões utilizadas.....16
Quadro 7 – Estrutura do repositório.....17
Quadro 8 – Desafios enfrentados e soluções adotadas.....27
Quadro 9 – Roteiro da apresentação.....43

LISTA DE ABREVIATURAS E SIGLAS

APM	Application Performance Monitoring
API	Application Programming Interface
CD	Continuous Delivery / Continuous Deployment
CI	Continuous Integration
CNCF	Cloud Native Computing Foundation
CPU	Central Processing Unit
DORA	DevOps Research and Assessment
ECS	Elastic Common Schema
GHCR	GitHub Container Registry
HPA	Horizontal Pod Autoscaler
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
K8s	Kubernetes
NDJSON	Newline Delimited JSON
RAM	Random Access Memory
SHA	Secure Hash Algorithm
SO	Sistema Operacional
VM	Virtual Machine (máquina virtual)
YAML	YAML Ain't Markup Language

SUMÁRIO

1 INTRODUÇÃO.....8
2 FUNDAMENTAÇÃO TEÓRICA: CONCEITOS FUNDAMENTAIS DE DEVOPS. .10
2.1 O QUE É DEVOPS E SUA RELAÇÃO COM AS METODOLOGIAS ÁGEIS.....10
2.2 DIFERENÇA ENTRE CI, CD (DELIVERY) E CD (DEPLOYMENT).....11
2.3 MÁQUINAS VIRTUAIS VS. CONTAINERS E VANTAGENS DOS CONTAINERS..12
2.4 ORQUESTRAÇÃO COM KUBERNETES E HELM.....14
2.5 LOGS, MÉTRICAS E APM COM A ELASTIC STACK.....14
3 METODOLOGIA, ARQUITETURA E FERRAMENTAS.....16
4 PASSO A PASSO DA IMPLEMENTAÇÃO.....18
4.1 APLICAÇÃO E TESTES AUTOMATIZADOS.....18
4.2 CONTAINERIZAÇÃO COM DOCKER.....19
4.3 PIPELINE DE INTEGRAÇÃO CONTÍNUA COM GITHUB ACTIONS (ETAPA 2)....19
4.4 CONTINUOUS DEPLOYMENT E ORQUESTRAÇÃO COM KUBERNETES E HELM (ETAPA 3)..... 20
4.5 LOGGING E MÉTRICAS COM FILEBEAT, ELASTICSEARCH E KIBANA (ETAPA 4)..... 23
4.6 AUTOMAÇÕES E RECURSOS EXTRAS.....25
5 DESAFIOS ENFRENTADOS.....27
6 REFLEXÃO CRÍTICA: COMO DEVOPS AGREGA VALOR AO CICLO DE VIDA DO SOFTWARE.....28
7 CONSIDERAÇÕES FINAIS.....30
REFERÊNCIAS.....31
APÊNDICE A – ARQUIVO .GITHUB/WORKFLOWS/MAIN.YML.....33
APÊNDICE B – DOCKERFILE E CHART HELM.....37
APÊNDICE C – CONFIGURAÇÃO DA ELASTIC STACK.....40
APÊNDICE D – ROTEIRO DA APRESENTAÇÃO ORAL (ATÉ 10 MINUTOS).....43

1 INTRODUÇÃO

A crescente demanda por entregas de software frequentes e confiáveis tornou insuficiente o modelo tradicional em que o desenvolvimento escreve o código e a operação, em um momento posterior e isolado, cuida de implantá-lo e mantê-lo em funcionamento. O movimento DevOps surge como resposta a esse problema, propondo uma cultura de colaboração entre as áreas e um conjunto de práticas fortemente apoiadas em automação, como integração contínua, entrega contínua, infraestrutura como código e monitoramento (KIM et al., 2018; SATO, 2013).

Este relatório documenta a Atividade Avaliativa – DevOps na Prática da disciplina DevOps Tools, cujo objetivo é consolidar os conceitos de DevOps e sua relação com as metodologias ágeis; implementar um pipeline de CI/CD com GitHub Actions; trabalhar com containers, registries e orquestração com Kubernetes e Helm; e implementar estratégias de monitoramento, logging e métricas (MARQUES, 2026a). Para isso, foi desenvolvida uma API de gerenciamento de tarefas em Node.js que percorre todo o ciclo: testes automatizados, empacotamento em container, publicação em um registry, implantação automática em Kubernetes e coleta de logs e métricas com a Elastic Stack.

A atividade foi dividida em cinco etapas, todas atendidas neste trabalho, conforme o Quadro 1.

Quadro 1 – Etapas da atividade e onde cada entrega é atendida

Etapa (peso)	Exigência	Onde está neste trabalho
1 – Conceitos fundamentais de DevOps (1,5)	Relatório: DevOps e Ágil; CI, CD (Delivery) e CD (Deployment); VMs vs. containers, com exemplos práticos ou analogias	Seção 2
2 – Integração e Entrega Contínua (2,0)	Repositório no GitHub com aplicação simples; pipeline que testa, faz build Docker e envia ao registry; arquivo .github/workflows/main.yml; print do pipeline com sucesso	Seções 4.1 a 4.3; Apêndice A
3 – Continuous Deployment e Orquestração (2,5)	Chart Helm; deploy automático no Kubernetes pelo pipeline; exposição via NodePort ou Ingress; diretório /helm; print do serviço no Kubernetes	Seção 4.4; Apêndice B
4 – Logging e Métricas (2,0)	Filebeat + Elasticsearch + Kibana; dashboard com logs em tempo real e métricas de requisições (status code e tempo de resposta); print; explicação da coleta pelos Beats	Seção 4.5; Apêndice C
5 – Relatório final e apresentação (2,0)	Passo a passo; desafios; reflexão crítica sobre o valor de DevOps; apresentação oral de até 10 minutos	Seções 4 a 6; Apêndice D

Fonte: Adaptado de Marques (2026a).

Todo o código-fonte, os manifestos e as configurações estão disponíveis no repositório https://github.com/[usuario]/[repositorio]. O restante do documento está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica (Etapa 1); a Seção 3 descreve a arquitetura e as ferramentas; a Seção 4 traz o passo a passo da implementação (Etapas 2, 3 e 4); a Seção 5 relata os desafios enfrentados; a Seção 6 apresenta a reflexão crítica; e a Seção 7, as considerações finais.

2 FUNDAMENTAÇÃO TEÓRICA: CONCEITOS FUNDAMENTAIS DE DEVOPS

Esta seção atende à Etapa 1 da atividade, explicando os conceitos de DevOps, integração e entrega contínuas e containers, sempre acompanhados de exemplos práticos retirados do próprio projeto e de analogias do cotidiano.

2.1 O QUE É DEVOPS E SUA RELAÇÃO COM AS METODOLOGIAS ÁGEIS

DevOps (junção de Development e Operations) é um conjunto de princípios culturais, práticas e ferramentas que aproxima as equipes de desenvolvimento e de operações para entregar software com mais rapidez, qualidade e estabilidade. Não se trata de um cargo ou de uma ferramenta específica, mas de uma forma de trabalhar em que a responsabilidade pelo software se estende do código até a produção. Kim et al. (2018) organizam essa forma de trabalhar em “três caminhos”: o fluxo (acelerar o trabalho do desenvolvimento até o cliente), o feedback (obter retorno rápido e contínuo de cada etapa) e o aprendizado contínuo (experimentar e melhorar sempre). Outra síntese bastante usada é o modelo CALMS: Cultura, Automação, Lean (enxuto), Medição e Compartilhamento (Sharing).

As metodologias ágeis, formalizadas no Manifesto Ágil (BECK et al., 2001), valorizam indivíduos e interações, software em funcionamento, colaboração com o cliente e resposta a mudanças, organizando o desenvolvimento em ciclos curtos (iterações ou sprints) com entregas incrementais. O ágil, porém, concentrou-se historicamente na relação entre o negócio e o time de desenvolvimento: de nada adianta produzir um incremento a cada duas semanas se a implantação em produção continua manual, demorada e arriscada. DevOps estende os princípios ágeis até a operação, removendo esse gargalo final (SATO, 2013). Assim, DevOps não substitui o ágil; complementa-o, conforme sintetiza o Quadro 2.

Quadro 2 – Relação entre metodologias ágeis e DevOps

Aspecto	Metodologias ágeis	DevOps
Foco principal	Colaboração entre negócio e desenvolvimento	Colaboração entre desenvolvimento e operações (e segurança)
Escopo	Planejamento e construção do software	Todo o ciclo: construção, teste, implantação, operação e monitoramento
Entrega	Incrementos ao final de cada iteração	Implantações frequentes e automatizadas, potencialmente várias por dia
Feedback	Revisões de sprint com o cliente	Testes automatizados, métricas e logs da aplicação em produção
Práticas típicas	Scrum, Kanban, histórias de usuário, retrospectivas	CI/CD, infraestrutura como código, containers, monitoramento

Fonte: Elaborado pelos autores (2026) com base em Beck et al. (2001) e Kim et al. (2018).

Analogia: em um restaurante, o ágil corresponde a uma cozinha que prepara pratos em pequenas porções e pede a opinião do cliente a cada rodada. Se, porém, o garçom (a operação) trabalhar isolado, os pratos esfriam no balcão. DevOps é colocar cozinha e salão como uma só equipe, com uma esteira que leva o prato ao cliente assim que fica pronto e um retorno imediato sobre o que agradou ou não.

Exemplo prático no projeto: cada alteração enviada ao GitHub dispara automaticamente testes, construção da imagem, publicação e implantação no Kubernetes; logo em seguida, o dashboard do Kibana mostra se a nova versão está respondendo com erros ou lentidão. O ciclo “alterar → entregar → observar” que, em um processo manual, levaria dias, acontece em poucos minutos.

2.2 DIFERENÇA ENTRE CI, CD (DELIVERY) E CD (DEPLOYMENT)

Integração Contínua (Continuous Integration – CI) é a prática de integrar o código de todos os desenvolvedores em um repositório compartilhado várias vezes ao dia, sendo cada integração verificada por um build e por testes automatizados, de modo que erros sejam detectados minutos após terem sido introduzidos (FOWLER, 2006). No projeto, a CI corresponde aos jobs que executam os testes com Jest e constroem a imagem Docker a cada push.

Entrega Contínua (Continuous Delivery) amplia a CI: além de testado, todo commit aprovado gera um artefato pronto para produção, e a implantação pode ser feita a qualquer momento, por decisão de negócio, normalmente com um clique (HUMBLE; FARLEY, 2014). Existe, portanto, uma aprovação manual antes da produção.

Implantação Contínua (Continuous Deployment) elimina essa aprovação manual: toda alteração que passa por todas as etapas do pipeline é implantada automaticamente em produção. Exige um conjunto de testes muito confiável e mecanismos de segurança, como rolling update e rollback automático. No projeto, o job de deploy aplica o chart Helm com a opção --atomic, que desfaz a atualização automaticamente se a nova versão não ficar saudável.

Quadro 3 – Comparação entre CI, Continuous Delivery e Continuous Deployment

Característica	CI	CD – Delivery	CD – Deployment
Objetivo	Detectar erros de integração cedo	Ter sempre uma versão pronta para produção	Levar cada mudança aprovada à produção automaticamente
Até onde automatiza	Build e testes	Build, testes e implantação em homologação	Todo o caminho até a produção
Decisão de ir para produção	Não se aplica	Manual (botão de aprovação)	Automática (sem intervenção humana)
Pré-requisito	Testes automatizados e repositório único	CI madura e ambientes padronizados	Testes confiáveis, monitoramento e rollback
No projeto	Jobs “Testes” e “Build Docker e push”	Imagem versionada no GHCR pronta para uso	Job “Deploy no Kubernetes (Helm)” a cada push na main

Fonte: Elaborado pelos autores (2026) com base em Fowler (2006) e Humble e Farley (2014).

Analogia: em uma fábrica de bolos, a CI é provar cada ingrediente e cada camada assim que ela é preparada, descobrindo logo se o açúcar foi trocado por sal. A Entrega Contínua é manter o bolo sempre pronto e embalado na vitrine, aguardando apenas que o gerente autorize a venda. A Implantação Contínua é a esteira que, ao terminar o controle de qualidade, coloca o bolo diretamente nas mãos do cliente, sem esperar autorização.

2.3 MÁQUINAS VIRTUAIS VS. CONTAINERS E VANTAGENS DOS CONTAINERS

Uma **máquina virtual (VM)** é uma cópia eficiente e isolada de uma máquina real, criada por um hipervisor que controla os recursos físicos (POPEK; GOLDBERG, 1974). Cada VM executa um sistema operacional completo — com o seu próprio kernel —, além das bibliotecas e da aplicação. Isso traz isolamento forte, mas também imagens de vários gigabytes, consumo elevado de memória e inicialização em minutos.

Um **container**, por sua vez, é um processo isolado que compartilha o kernel do sistema operacional hospedeiro, utilizando recursos do próprio Linux, como namespaces (isolamento de processos, rede e sistema de arquivos) e cgroups (limites de CPU e memória). A imagem do container carrega apenas a aplicação e suas dependências, o que a torna leve,

portátil e rápida de iniciar (MERKEL, 2014; DOCKER INC., 2026). O Quadro 4 resume as diferenças.

Quadro 4 – Comparação entre máquinas virtuais e containers

Critério	Máquina virtual	Container
Virtualiza	O hardware (via hipervisor)	O sistema operacional (via namespaces e cgroups)
Sistema operacional	Um SO completo por VM	Compartilha o kernel do hospedeiro
Tamanho típico	Gigabytes	Megabytes (a imagem deste projeto tem cerca de 227 MB)
Inicialização	Minutos	Segundos ou menos
Densidade	Poucas VMs por servidor	Dezenas ou centenas de containers por servidor
Isolamento	Forte (kernel separado)	Bom, porém dependente do kernel compartilhado
Uso típico	Sistemas operacionais diferentes, isolamento rígido	Microsserviços, CI/CD, ambientes reprodutíveis

Fonte: Elaborado pelos autores (2026) com base em Popek e Goldberg (1974), Merkel (2014) e Docker Inc. (2026).

As principais vantagens dos containers são: **portabilidade**, pois a mesma imagem roda no computador do desenvolvedor, no runner do GitHub Actions e no cluster Kubernetes, eliminando o problema do “na minha máquina funciona”; **leveza e velocidade**, que permitem subir e destruir ambientes em segundos; **imutabilidade e versionamento**, já que cada imagem é identificada por uma tag (neste projeto, o SHA do commit), facilitando rastrear e reverter versões; **eficiência de recursos**, com mais aplicações por servidor; e **base para a orquestração**, permitindo que ferramentas como o Kubernetes escalem, reiniciem e atualizem a aplicação automaticamente.

Analogia: uma VM é como uma casa: tem fundação, encanamento e estrutura próprios, o que a torna muito isolada, mas cara e demorada de construir. Containers são como apartamentos de um mesmo prédio: compartilham a fundação e a estrutura (o kernel), mas cada um tem sua porta, sua chave e seus móveis (sistema de arquivos e processos isolados), sendo muito mais rápidos de ocupar.

Exemplo prático no projeto: durante o desenvolvimento, Elasticsearch, Kibana, Filebeat, a aplicação e um gerador de tráfego — cinco serviços — foram executados simultaneamente com um único comando (docker compose up) em um notebook com 8 GB de RAM, algo inviável com cinco máquinas virtuais completas.

2.4 ORQUESTRAÇÃO COM KUBERNETES E HELM

Kubernetes é uma plataforma de código aberto, mantida pela Cloud Native Computing Foundation (CNCF), que automatiza a implantação, a escala e o gerenciamento de aplicações em containers. O usuário declara o estado desejado — por exemplo, “duas réplicas da imagem X expostas na porta 30080” — e o Kubernetes trabalha continuamente para mantê-lo, recriando pods que falham e substituindo versões de forma gradual (BURNS; BEDA; HIGHTOWER, 2019; THE KUBERNETES AUTHORS, 2026). Os objetos usados neste projeto são: Deployment (gerencia as réplicas e as atualizações), Pod (unidade que executa o container), Service (endereço estável e balanceamento entre os pods; o tipo NodePort expõe a aplicação fora do cluster) e ConfigMap (configurações injetadas como variáveis de ambiente).

O Helm é o gerenciador de pacotes do Kubernetes. Um chart reúne modelos (templates) de manifestos parametrizados por um arquivo values.yaml, permitindo instalar, atualizar, versionar e reverter uma aplicação com um único comando (HELM, 2026). No pipeline, o mesmo chart é reutilizado a cada versão, alterando apenas a tag da imagem.

2.5 LOGS, MÉTRICAS E APM COM A ELASTIC STACK

Logs são registros gerados pelas aplicações e serviços com informações sobre eventos, erros, execuções e estados da aplicação; seu gerenciamento centralizado viabiliza diagnóstico de erros, auditoria e segurança, monitoramento de desempenho e análise de comportamento. Já o APM (Application Performance Monitoring) é o conjunto de práticas e ferramentas que monitora disponibilidade, tempo de resposta, taxas de erro e outras métricas-chave de desempenho (MARQUES, 2026b). Beyer et al. (2016) destacam, entre os “sinais de ouro” do monitoramento, a latência, o tráfego e os erros — exatamente os indicadores exibidos no dashboard deste projeto.

A Elastic Stack é o conjunto de ferramentas da Elastic para coleta, armazenamento, busca e visualização de logs e métricas: os Beats são agentes leves de coleta; o Elasticsearch é o motor de busca e indexação; o Kibana é a interface de visualização e análise; e o Logstash, opcional, transforma os dados antes do armazenamento (ELASTIC, 2026a; MARQUES, 2026b). O Quadro 5 apresenta os principais Beats.

Quadro 5 – Principais Beats da Elastic Stack

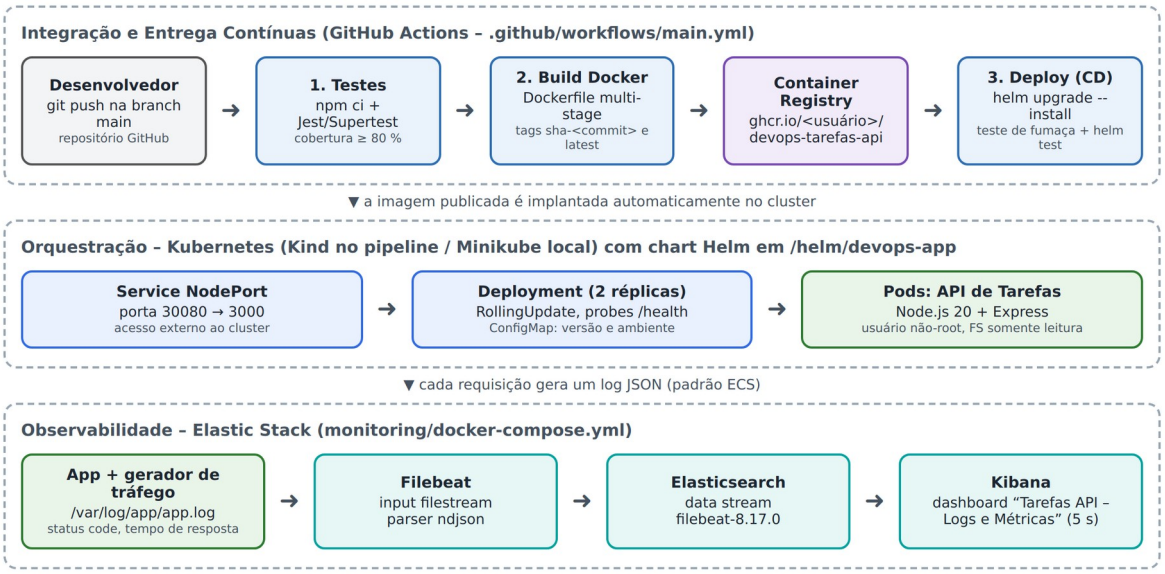
Beat	Função	Uso neste projeto
Filebeat	Coleta e envia logs	Sim: lê o arquivo de log JSON da API
Metricbeat	Coleta métricas de sistema e de aplicações	Não (métricas de requisição extraídas dos logs)
Packetbeat	Coleta dados de rede	Não
Auditbeat	Monitora alterações de segurança	Não

Fonte: Adaptado de Marques (2026b).

3 METODOLOGIA, ARQUITETURA E FERRAMENTAS

O projeto foi desenvolvido de forma incremental, seguindo a ordem das etapas da atividade: primeiro a aplicação e seus testes, depois a containerização e o pipeline de CI, em seguida o chart Helm e o deploy automatizado e, por fim, a pilha de observabilidade. Cada componente foi validado localmente antes de ser integrado ao pipeline. A Figura 1 apresenta a arquitetura completa da solução.

Figura 1 – Arquitetura da solução: CI/CD, orquestração e observabilidade



Fonte: Elaborado pelos autores (2026).

O Quadro 6 lista as ferramentas e versões utilizadas, e o Quadro 7 apresenta a estrutura do repositório.

Quadro 6 – Ferramentas e versões utilizadas

Ferramenta	Versão	Papel no projeto
Node.js / Express	20 (imagem) / 4.21.2	Linguagem e framework da API de tarefas
Jest / Supertest	29.7.0 / 7.1.4	Testes automatizados e cobertura de código
Docker	29.9.2	Construção e execução de containers
GitHub Actions	–	Orquestração do pipeline de CI/CD
GitHub Container Registry	–	Registro das imagens Docker (ghcr.io)
Kubernetes (Kind / Minikube)	Kind v1.12 action / Minikube 1.39	Cluster no pipeline / cluster local
Helm	3.22.0	Empacotamento e implantação da aplicação no Kubernetes
Elasticsearch / Kibana / Filebeat	8.17.0	Armazenamento, visualização e coleta de logs

Fonte: Elaborado pelos autores (2026).

Quadro 7 – Estrutura do repositório

```
.
├── .github/workflows/main.yml   # pipeline CI/CD (GitHub Actions)
├── app/                         # API Node.js
│   ├── src/ (app.js, server.js, logger.js)
│   ├── public/index.html        # interface web
│   ├── test/app.test.js         # testes automatizados
│   └── Dockerfile
├── helm/devops-app/             # chart Helm (Etapa 3)
│   ├── Chart.yaml, values.yaml
│   └── templates/ (deployment, service, configmap, hpa, ingress, tests)
├── k8s/kind-config.yaml         # cluster Kind usado no pipeline
├── monitoring/                  # Elastic Stack (Etapa 4)
│   ├── docker-compose.yml
│   ├── filebeat/filebeat.yml
│   ├── gerar-trafego.sh
│   └── kibana/ (dashboard-tarefas.ndjson, importar-dashboard.sh)
├── docs/                        # relatório e prints
└── README.md
```

Fonte: Elaborado pelos autores (2026).

4 PASSO A PASSO DA IMPLEMENTAÇÃO

Esta seção descreve, em ordem cronológica, como cada parte do projeto foi construída e validada, atendendo às Etapas 2, 3 e 4 e ao item “passo a passo da implementação” da Etapa 5.

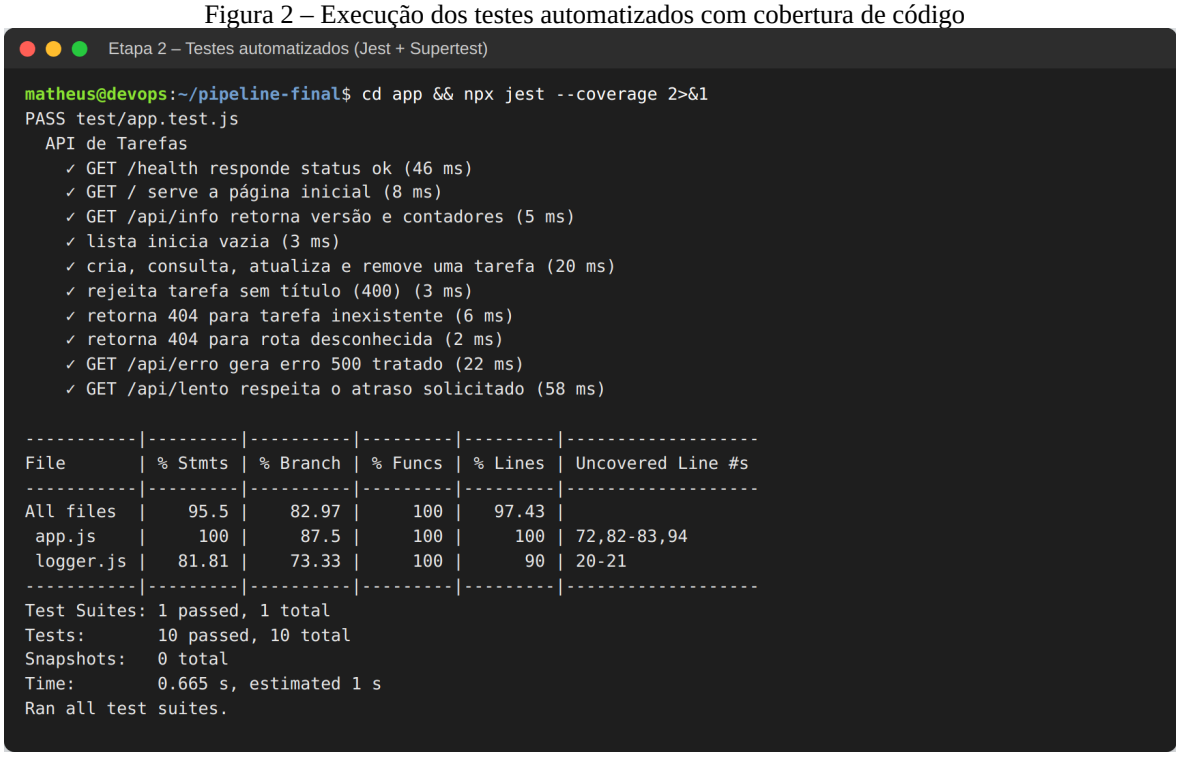
4.1 APLICAÇÃO E TESTES AUTOMATIZADOS

Passo 1 – Criação da aplicação. Foi desenvolvida uma API REST de tarefas em Node.js com Express (diretório app/), com as rotas: GET / (interface web), GET /health (verificação de saúde usada pelo Kubernetes), GET /api/info (versão, ambiente, nome do pod e contadores), CRUD completo em /api/tarefas e duas rotas de demonstração para o monitoramento: /api/lento, que responde com atraso de 300 ms a 1,2 s, e /api/erro, que gera um erro 500 tratado.

Passo 2 – Logs estruturados. Um middleware registra cada requisição em uma linha JSON (formato NDJSON) com campos do Elastic Common Schema (ELASTIC, 2026c): @timestamp, log.level, http.request.method, url.path, http.response.status_code, event.duration (nanossegundos) e o campo próprio response_time_ms. Respostas 4xx geram nível warn e 5xx, nível error. Os logs vão para a saída padrão e, quando definida a variável LOG_FILE, também para um arquivo, que é lido pelo Filebeat.

Passo 3 – Testes automatizados. Foram escritos dez testes com Jest e Supertest cobrindo health check, página inicial, criação, consulta, atualização e remoção de tarefas, validação (400), recursos inexistentes (404), erro tratado (500) e a rota lenta. O projeto exige cobertura mínima de 80% de linhas; se ela não for atingida, o pipeline falha. A Figura 2 mostra a execução local: 10 testes aprovados e 97,4% de cobertura de linhas.

Figura 2 – Execução dos testes automatizados com cobertura de código



Fonte: Elaborado pelos autores (2026).

4.2 CONTAINERIZAÇÃO COM DOCKER

Passo 4 – Dockerfile. A imagem usa build em duas etapas (multi-stage): a primeira instala apenas as dependências de produção com npm ci --omit=dev; a segunda copia somente o necessário para uma imagem node:20-alpine, executa o processo com o usuário sem privilégios node, expõe a porta 3000 e define um HEALTHCHECK que consulta /health. O arquivo .dockerignore impede que testes, node_modules e logs locais entrem na imagem. O Dockerfile completo está no Apêndice B.

Passo 5 – Validação local. A imagem foi construída com docker build -t devops-tarefas-api:local app, resultando em cerca de 227 MB, e executada com sucesso tanto isoladamente quanto no Docker Compose da Etapa 4.

4.3 PIPELINE DE INTEGRAÇÃO CONTÍNUA COM GITHUB ACTIONS (ETAPA 2)

Passo 6 – Repositório e workflow. O código foi versionado em um repositório público no GitHub e o pipeline foi definido no arquivo .github/workflows/main.yml (Apêndice A), executado a cada push ou pull request na branch main e também manualmente (workflow_dispatch). Os jobs são encadeados com needs, de modo que uma etapa só é executada se a anterior tiver sucesso:

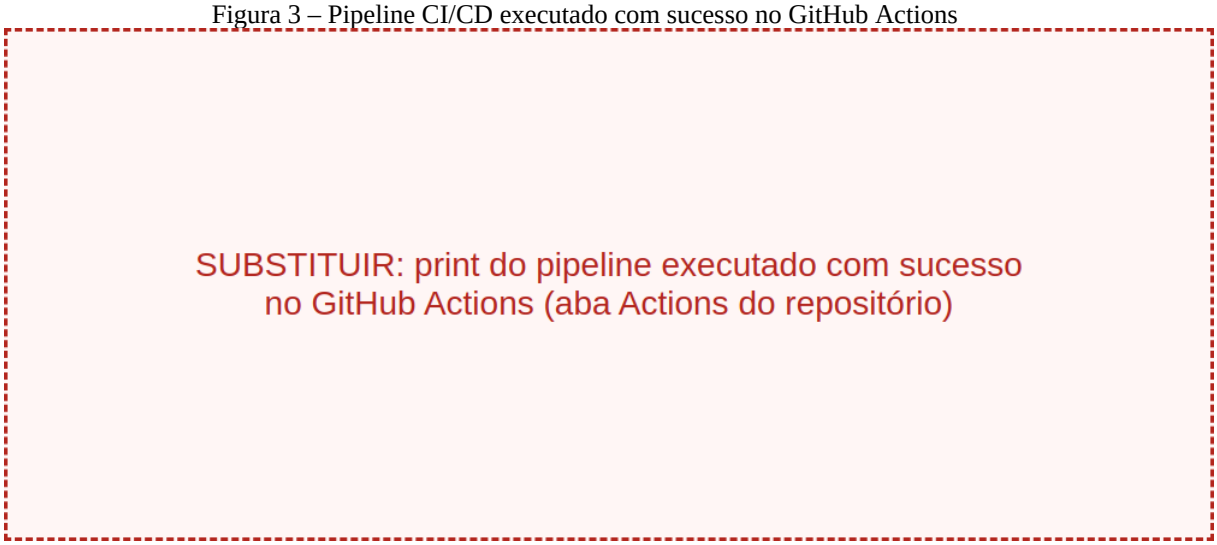
– **Job 1 – Testes automatizados:** faz checkout, instala o Node.js 20 com cache do npm, executa npm ci e npm test (Jest com limite de cobertura) e publica o relatório de cobertura como artefato;

– **Job 2 – Build Docker e push (GHCR):** define o nome da imagem em letras minúsculas, autentica no GitHub Container Registry com o GITHUB_TOKEN automático (sem necessidade de cadastrar segredos), gera as tags sha-<commit> e latest com docker/metadata-action e constrói e envia a imagem com docker/build-push-action, usando cache do GitHub Actions para acelerar builds subsequentes. Em pull requests, a imagem é construída, mas não publicada;

– **Job 3 – Deploy no Kubernetes (Helm):** descrito na Seção 4.4.

Passo 7 – Execução. Após o push, o pipeline foi executado com sucesso, conforme a Figura 3. A imagem publicada fica disponível na seção Packages do repositório, com a tag correspondente ao commit.

Figura 3 – Pipeline CI/CD executado com sucesso no GitHub Actions



Fonte: Elaborado pelos autores (2026).

4.4 CONTINUOUS DEPLOYMENT E ORQUESTRAÇÃO COM KUBERNETES E HELM (ETAPA 3)

Passo 8 – Chart Helm. No diretório /helm foi criado o chart devops-app (Apêndice B), composto por:

– **Chart.yaml:** metadados e versão do chart;

– **values.yaml:** parâmetros padrão — 2 réplicas, repositório e tag da imagem, Service do tipo NodePort na porta 30080, requisições e limites de CPU e memória e contexto de segurança;

– **templates/deployment.yaml:** Deployment com estratégia RollingUpdate (maxSurge 1, maxUnavailable 0, garantindo atualização sem indisponibilidade), readinessProbe e livenessProbe em /health, execução como usuário não-root com sistema de arquivos somente leitura e anotação de checksum que força novo rollout quando a configuração muda;
– **templates/service.yaml:** Service NodePort que expõe a aplicação fora do cluster (porta 80 → 3000 do container, nodePort 30080);
– **templates/configmap.yaml:** variáveis de ambiente (porta, ambiente e versão — esta última igual à tag da imagem, o que permite confirmar qual versão está no ar);
– **templates/hpa.yaml e ingress.yaml:** autoescalonamento horizontal e Ingress, opcionais e desativados por padrão;
– **templates/tests/test-connection.yaml:** teste executado com helm test, que chama o /health através do Service.

Passo 9 – Deploy automático no pipeline (CD). O terceiro job do workflow é executado após a publicação da imagem. Ele cria um cluster Kubernetes efêmero com Kind dentro do runner (com o NodePort 30080 mapeado para o host), cria o namespace devops e o segredo ghcr-cred para baixar a imagem do registry, valida o chart com helm lint e executa helm upgrade --install com a nova tag, usando as opções --wait e --atomic (se a nova versão não ficar pronta, o Helm desfaz a atualização automaticamente). Em seguida, verifica o rollout, lista deployments, pods e services, faz um teste de fumaça via NodePort — chamando /health e conferindo se /api/info retorna exatamente a tag recém-publicada — e roda helm test. Dessa forma, cada nova versão da aplicação é publicada no cluster automaticamente, sem intervenção humana, caracterizando Continuous Deployment.

Como o cluster Kind do runner é temporário, o workflow inclui ainda um quarto job opcional, deploy-minikube, que aplica o mesmo chart em um cluster Minikube persistente por meio de um runner self-hosted; ele é ativado criando a variável de repositório DEPLOY_MINIKUBE=true.

Passo 10 – Validação no Minikube. O mesmo chart foi implantado no Minikube local. A Figura 4 mostra o release instalado, o Deployment com 2/2 réplicas prontas, os pods em execução e o Service do tipo NodePort expondo a porta 30080, além das respostas de /health e /api/info obtidas de fora do cluster.

Figura 4 – Serviço da aplicação rodando no Kubernetes (Minikube) e exposto via NodePort



Fonte: Elaborado pelos autores (2026).

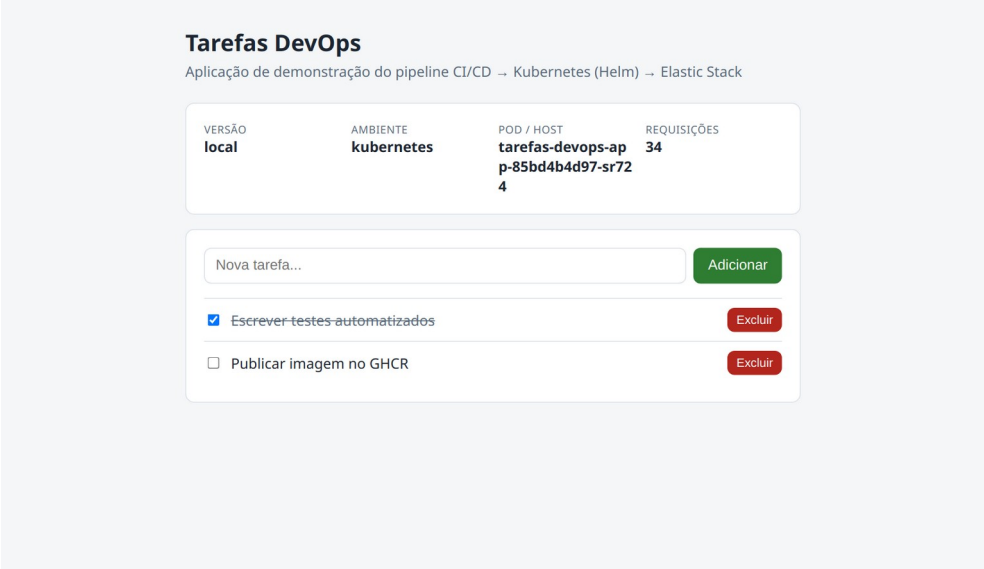
A Figura 5 mostra a validação do chart com helm lint e a execução do teste do release com helm test, e a Figura 6 mostra a aplicação acessada pelo navegador no endereço do NodePort, exibindo o nome do pod que atendeu à requisição.

Figura 5 – Validação do chart (helm lint) e teste do release (helm test)



Fonte: Elaborado pelos autores (2026).

Figura 6 – Aplicação acessada pelo navegador por meio do NodePort 30080



Fonte: Elaborado pelos autores (2026).

4.5 LOGGING E MÉTRICAS COM FILEBEAT, ELASTICSEARCH E KIBANA (ETAPA 4)

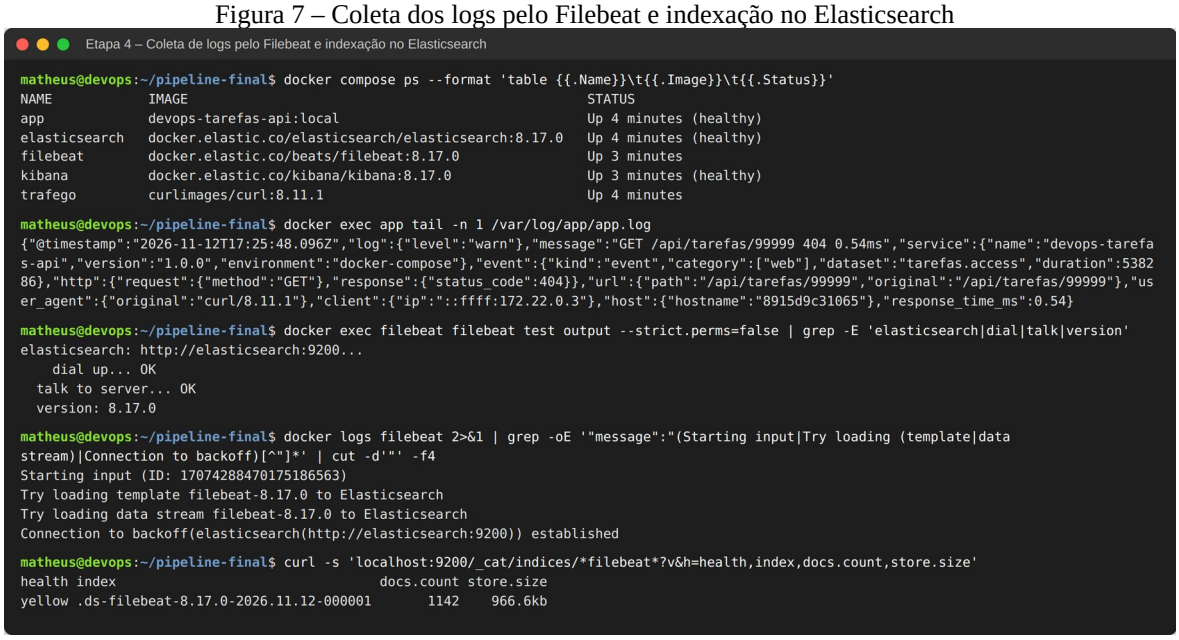
Passo 11 – Elastic Stack em containers. O arquivo monitoring/docker-compose.yml (Apêndice C) sobe cinco serviços: Elasticsearch 8.17 em modo single-node, com heap limitado a 512 MB e segurança desativada (ambiente acadêmico); Kibana 8.17; a aplicação, configurada com LOG_FILE=/var/log/app/app.log; o Filebeat 8.17; e um gerador de tráfego (script com curl) que produz requisições contínuas e variadas — respostas 200, 201, 204, 400, 404, 500 e respostas lentas — para alimentar o dashboard. Healthchecks garantem a ordem de inicialização: Filebeat e Kibana só sobem após o Elasticsearch estar saudável.

Passo 12 – Como os Beats coletaram os dados. A coleta segue o fluxo apresentado na aula: Filebeat lê os logs → envia ao Elasticsearch → dados disponíveis no Kibana (MARQUES, 2026b). Em detalhes:

- **Geração:** a cada requisição, a API grava uma linha JSON no arquivo /var/log/app/app.log, localizado em um volume Docker (app-logs) compartilhado com o container do Filebeat em modo somente leitura;
- **Leitura (input filestream):** o Filebeat monitora o caminho /var/log/app/*.log. Para cada arquivo encontrado, ele inicia um leitor (harvester) que acompanha o arquivo linha a linha, como um tail -f, e guarda a posição lida (offset) em um registro interno (registry), persistido no volume filebeat-data. Assim, se o Filebeat reiniciar, ele continua de onde parou, sem perder nem duplicar eventos;
- **Decodificação (parser ndjson):** cada linha é interpretada como JSON e seus campos são colocados na raiz do evento (target: ""), de modo que http.response.status_code e response_time_ms se tornam campos pesquisáveis, e não apenas texto. O Filebeat acrescenta metadados como log.file.path, agent.* e host.*, além do campo origem;
- **Envio (output.elasticsearch):** os eventos são enviados em lotes (bulk) ao Elasticsearch, no data stream filebeat-8.17.0. Na primeira conexão, o Filebeat instala o index template com os tipos ECS — status code como número inteiro — e, por meio de setup.template.append_fields, o campo response_time_ms como número decimal (float). Em caso de indisponibilidade do Elasticsearch, o Filebeat aplica backoff e reenvia, garantindo entrega ao menos uma vez;
- **Visualização:** o Kibana consulta o data stream por meio do data view filebeat-*, usando @timestamp como campo de tempo.

A Figura 7 evidencia esse fluxo: os containers em execução, uma linha de log JSON gerada pela aplicação, o teste de conexão do Filebeat com o Elasticsearch, as mensagens de inicialização do input e de carga do template e o índice com os documentos já indexados.

Figura 7 – Coleta dos logs pelo Filebeat e indexação no Elasticsearch



Fonte: Elaborado pelos autores (2026).

Passo 13 – Dashboard no Kibana. Em vez de criar o dashboard apenas manualmente, ele foi definido como código: o script monitoring/kibana/gerar_dashboard.py gera o arquivo dashboard-tarefas.ndjson com o data view, a busca salva e as visualizações, e o script importar-dashboard.sh o importa pela API de saved objects do Kibana. Assim, o dashboard é versionado no Git e pode ser recriado em qualquer ambiente com um comando. O dashboard “Tarefas API – Logs e Métricas” (Figura 8) contém:

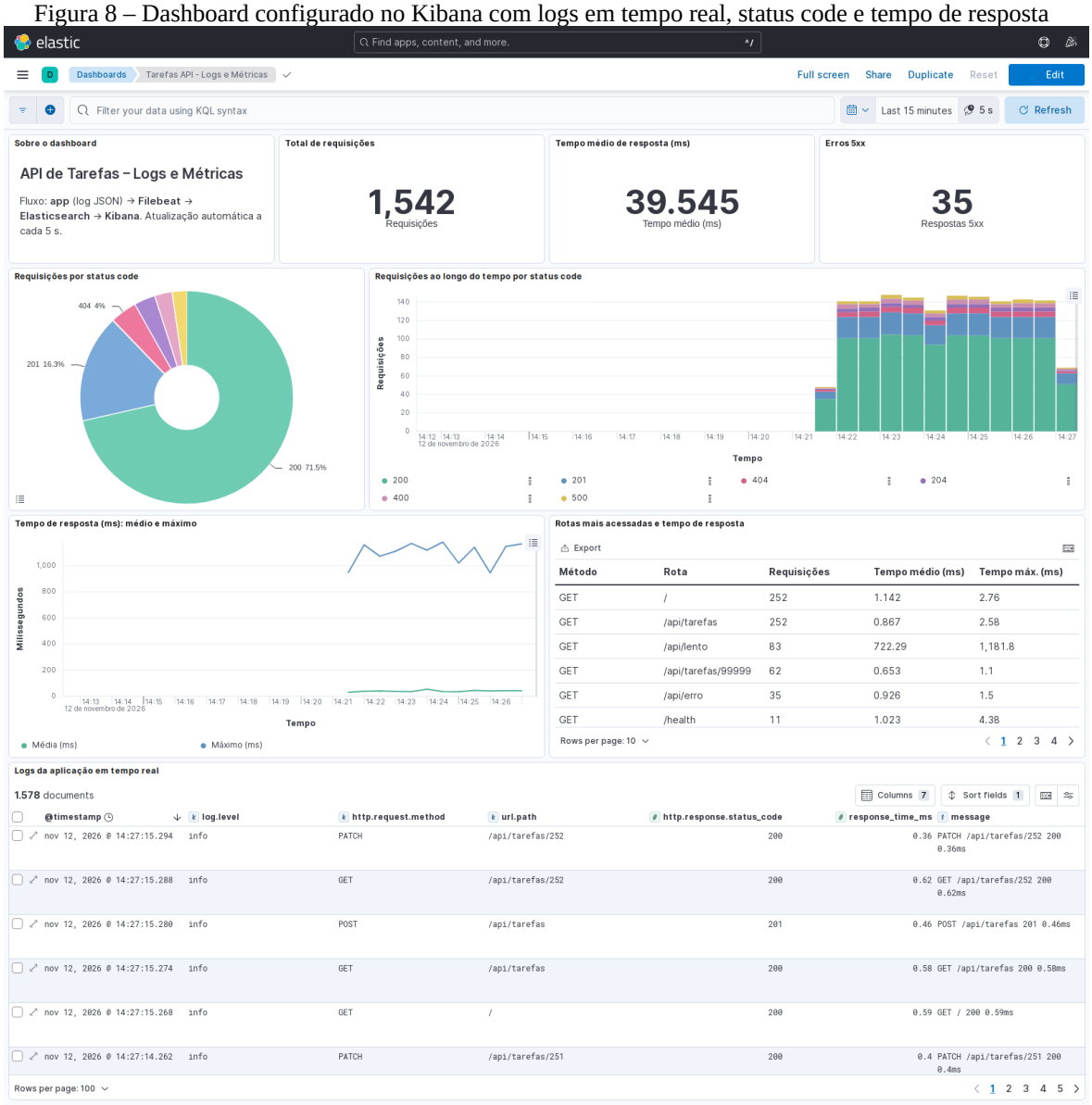
– **Logs da aplicação em tempo real:** tabela com data/hora, nível, método, rota, status code, tempo de resposta e mensagem, ordenada do mais recente para o mais antigo, com atualização automática a cada 5 segundos e janela dos últimos 15 minutos;

– **Métricas de status code:** gráfico de rosca com a distribuição percentual dos códigos HTTP e gráfico de barras empilhadas com a quantidade de requisições por status ao longo do tempo;

– **Métricas de tempo de resposta:** indicador de tempo médio, gráfico de linhas com os tempos médio e máximo ao longo do tempo e tabela de rotas mais acessadas com tempos médio e máximo — evidenciando, por exemplo, que /api/lento responde em cerca de 720 ms em média, enquanto as demais rotas respondem em cerca de 1 ms;

– **Indicadores-resumo:** total de requisições e quantidade de erros 5xx.

Figura 8 – Dashboard configurado no Kibana com logs em tempo real, status code e tempo de resposta



Fonte: Elaborado pelos autores (2026).

4.6 AUTOMAÇÕES E RECURSOS EXTRAS

Além do exigido, foram implementados os seguintes recursos, que buscam atender ao critério de criatividade da atividade:

– aplicação com interface web própria, que exibe a versão implantada e o pod que respondeu (útil para demonstrar o balanceamento entre réplicas e a troca de versão);

– teste de fumaça automatizado no pipeline, que confirma se a versão no ar é exatamente a do commit, e helm test;

– deploy atômico com rollback automático (--atomic) e rolling update sem indisponibilidade;

– dashboard do Kibana versionado como código e importado por script;

– gerador de tráfego para demonstração contínua do monitoramento;
– resumo do pipeline (job summary) com a imagem publicada e os pods em execução;
– job opcional de deploy em Minikube via runner self-hosted;
– suporte opcional ao Elastic APM: o agente elastic-apm-node é ativado automaticamente quando a variável ELASTIC_APM_SERVER_URL é definida, conforme o passo extra sugerido na Aula 5;
– boas práticas de segurança: container sem privilégios, sistema de arquivos somente leitura, capabilities removidas e autenticação no registry sem senhas fixas.

5 DESAFIOS ENFRENTADOS

Durante a implementação surgiram problemas reais, resumidos no Quadro 8 e comentados a seguir. Registrá-los é, em si, uma prática DevOps: cada falha vira aprendizado e melhoria no processo.

Quadro 8 – Desafios enfrentados e soluções adotadas

Desafio	Causa	Solução
helm test falhou com “bad address” logo após iniciar o Minikube	O CoreDNS ainda não estava pronto, então o nome do Service não era resolvido	Aguardar o CoreDNS ficar Ready (kubectl wait) antes dos testes
helm test --logs não exibia a saída do teste	A política hook-succeeded apagava o pod do teste antes da leitura dos logs	Manter apenas before-hook-creation na política de remoção
Lista de tarefas “mudava” entre atualizações da página	Com 2 réplicas, cada pod guarda seus dados em memória e o Service alterna entre eles	Documentado como limitação; em produção, usar banco de dados externo (aplicação stateless)
Risco de response_time_ms ser mapeado como inteiro	O mapeamento dinâmico do Elasticsearch usa o tipo do primeiro valor recebido	Declarar o campo como float em setup.template.append_fields
Nome da imagem rejeitado pelo registry	O GHCR exige letras minúsculas e o usuário do GitHub pode ter maiúsculas	Converter o nome com ${GITHUB_REPOSITORY_OWNER,,}
Cluster sem permissão para baixar a imagem	Pacotes novos no GHCR são privados por padrão	Criar o segredo ghcr-cred com o GITHUB_TOKEN e usar imagePullSecrets
Memória insuficiente para tudo ao mesmo tempo	Notebook com 8 GB de RAM para Minikube e Elastic Stack	Limitar o heap do Elasticsearch a 512 MB e executar Minikube e Elastic Stack em momentos distintos
Cluster do pipeline não é persistente	O Kind é criado dentro do runner e destruído ao final do job	Validar a implantação no próprio job (teste de fumaça) e oferecer job opcional com runner self-hosted

Fonte: Elaborado pelos autores (2026).

O desafio mais instrutivo foi o comportamento da lista de tarefas com duas réplicas: ao atualizar a página, ora apareciam algumas tarefas, ora outras. A causa é que cada pod mantém seu próprio estado em memória, e o Service distribui as requisições entre eles. O episódio ilustra na prática um princípio da computação em nuvem: aplicações que escalam horizontalmente devem ser stateless, mantendo o estado em serviços externos, como bancos de dados. Outro aprendizado importante veio da falha intermitente do helm test: um teste que falha por condição de corrida (DNS ainda não pronto) mostra por que pipelines precisam aguardar explicitamente que as dependências estejam saudáveis — razão pela qual o workflow usa --wait, rollout status e tentativas repetidas no teste de fumaça.

6 REFLEXÃO CRÍTICA: COMO DEVOPS AGREGA VALOR AO CICLO DE VIDA DO SOFTWARE

A experiência deste projeto permite observar, em pequena escala, os ganhos que DevOps traz a cada fase do ciclo de vida do software. No **desenvolvimento**, os testes automatizados executados a cada push funcionam como uma rede de segurança: um erro é apontado em poucos minutos e associado ao commit que o introduziu, quando ainda é barato corrigi-lo — o chamado shift-left. Na **construção e entrega**, a imagem Docker imutável e versionada pelo SHA do commit garante que o artefato testado é exatamente o mesmo implantado, eliminando divergências entre ambientes. Na **implantação**, o Helm com rolling update e rollback automático transforma o deploy — tradicionalmente um evento tenso, manual e feito fora do horário comercial — em uma operação rotineira e reversível. Na **operação**, os logs estruturados e o dashboard tornam visível o que acontece em produção: a rota lenta e os erros 500 simulados aparecem imediatamente nos gráficos, fechando o ciclo de feedback para o time.

Esses ganhos são confirmados pela pesquisa do grupo DevOps Research and Assessment (DORA), que associa práticas como integração e entrega contínuas, automação de testes e monitoramento a melhor desempenho em quatro métricas: frequência de implantação, tempo de espera para mudanças, tempo de restauração do serviço e taxa de falhas em mudanças (FORSGREN; HUMBLE; KIM, 2018). Neste projeto, o tempo entre um commit e a nova versão em execução no cluster caiu para alguns minutos, e o rollback automático reduz o impacto de uma mudança defeituosa.

É necessário, porém, olhar criticamente para essas vantagens. Primeiro, ferramentas não produzem DevOps sozinhas: sem colaboração entre as pessoas, responsabilidade compartilhada e cultura de aprendizado sem culpados, o pipeline apenas automatiza um processo ruim mais rapidamente (KIM et al., 2018). Segundo, a automação introduz complexidade e custo próprios: Kubernetes, Helm e Elastic Stack exigem conhecimento especializado e recursos computacionais consideráveis — a Elastic Stack, sozinha, consumiu a maior parte da memória disponível no ambiente de desenvolvimento —, e para sistemas pequenos uma solução mais simples pode ser mais adequada. Terceiro, a Implantação Contínua só é segura com testes de qualidade: a cobertura de 97% deste projeto mede quantas linhas foram executadas, não se todos os cenários relevantes foram verificados. Por fim, simplificações aceitáveis em ambiente acadêmico — como a segurança do Elasticsearch desativada e um cluster efêmero no pipeline — precisariam ser revistas em produção, com

autenticação, TLS, gestão de segredos, varredura de vulnerabilidades nas imagens (DevSecOps) e ambientes de homologação.

Conclui-se que DevOps agrega valor principalmente ao encurtar e tornar confiável o caminho entre uma ideia e o usuário, e ao devolver ao time informações reais sobre o software em execução. O valor não está em uma ferramenta isolada, mas na integração de práticas que tornam as entregas pequenas, frequentes, observáveis e reversíveis.

7 CONSIDERAÇÕES FINAIS

Os objetivos da atividade foram atingidos. Na Etapa 1, os conceitos de DevOps, metodologias ágeis, CI, Continuous Delivery, Continuous Deployment, máquinas virtuais e containers foram explicados com exemplos e analogias. Na Etapa 2, construiu-se uma aplicação Node.js com dez testes automatizados e um pipeline no GitHub Actions que executa os testes, constrói a imagem Docker e a publica no GitHub Container Registry. Na Etapa 3, um chart Helm permitiu que o mesmo pipeline implantasse automaticamente cada nova versão em um cluster Kubernetes, exposta via NodePort e validada por testes de fumaça. Na Etapa 4, Filebeat, Elasticsearch e Kibana coletaram os logs estruturados da aplicação e alimentaram um dashboard com logs em tempo real, status code e tempo de resposta. Na Etapa 5, foram documentados o passo a passo, os desafios e uma reflexão crítica, e preparou-se o roteiro da apresentação (Apêndice D).

Como trabalhos futuros, sugerem-se: persistir as tarefas em um banco de dados para tornar a aplicação stateless; adicionar um ambiente de homologação com aprovação manual, combinando Continuous Delivery e Continuous Deployment; incluir varredura de vulnerabilidades da imagem no pipeline; coletar os logs diretamente do cluster Kubernetes com Filebeat em DaemonSet; e ativar o Elastic APM com APM Server para rastreamento distribuído das transações.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BECK, Kent et al. **Manifesto para desenvolvimento ágil de software**. [S. l.], 2001. Disponível em: https://agilemanifesto.org/iso/ptbr/manifesto.html. Acesso em: 12 nov. 2026.

BEYER, Betsy et al. **Site reliability engineering**: how Google runs production systems. Sebastopol: O'Reilly Media, 2016.

BURNS, Brendan; BEDA, Joe; HIGHTOWER, Kelsey. **Kubernetes**: up and running: dive into the future of infrastructure. 2. ed. Sebastopol: O'Reilly Media, 2019.

DOCKER INC. **What is a container?** [S. l.], 2026. Disponível em: https://www.docker.com/resources/what-container/. Acesso em: 12 nov. 2026.

ELASTIC. **Elastic documentation**. [S. l.], 2026a. Disponível em: https://www.elastic.co/guide/. Acesso em: 12 nov. 2026.

ELASTIC. **Filebeat reference [8.17]**. [S. l.], 2026b. Disponível em: https://www.elastic.co/guide/en/beats/filebeat/8.17/index.html. Acesso em: 12 nov. 2026.

ELASTIC. **Elastic Common Schema (ECS) reference**. [S. l.], 2026c. Disponível em: https://www.elastic.co/guide/en/ecs/current/index.html. Acesso em: 12 nov. 2026.

FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. **Accelerate**: the science of lean software and DevOps: building and scaling high performing technology organizations. Portland: IT Revolution Press, 2018.

FOWLER, Martin. **Continuous integration**. [S. l.], 2006. Disponível em: https://martinfowler.com/articles/continuousIntegration.html. Acesso em: 12 nov. 2026.

GITHUB. **GitHub Actions documentation**. [S. l.], 2026. Disponível em: https://docs.github.com/actions. Acesso em: 12 nov. 2026.

HELM. **Helm documentation**. [S. l.], 2026. Disponível em: https://helm.sh/docs/. Acesso em: 12 nov. 2026.

HUMBLE, Jez; FARLEY, David. **Entrega contínua**: como entregar software de forma rápida e confiável. Porto Alegre: Bookman, 2014.

KIM, Gene et al. **Manual de DevOps**: como obter agilidade, confiabilidade e segurança em organizações tecnológicas. Rio de Janeiro: Alta Books, 2018.

MARQUES, Rodrigo Silva. **Atividade avaliativa**: DevOps na prática. Belém: Universidade da Amazônia, 2026a. Material didático da disciplina DevOps Tools.

MARQUES, Rodrigo Silva. **Logs e Application Performance Monitoring (APM) com Elastic Stack**: aula 5. Belém: Universidade da Amazônia, 2026b. Material didático da disciplina DevOps Tools.

MERKEL, Dirk. Docker: lightweight Linux containers for consistent development and deployment. **Linux Journal**, Houston, v. 2014, n. 239, mar. 2014.

POPEK, Gerald J.; GOLDBERG, Robert P. Formal requirements for virtualizable third generation architectures. **Communications of the ACM**, New York, v. 17, n. 7, p. 412-421, jul. 1974.

SATO, Danilo. **DevOps na prática**: entrega de software confiável e automatizada. São Paulo: Casa do Código, 2013.

THE KUBERNETES AUTHORS. **Kubernetes documentation**. [S. l.], 2026. Disponível em: https://kubernetes.io/docs/home/. Acesso em: 12 nov. 2026.

APÊNDICE A – ARQUIVO .GITHUB/WORKFLOWS/MAIN.YML

Pipeline completo de CI/CD utilizado no projeto (Etapas 2 e 3).

```
# Pipeline CI/CD - Atividade Avaliativa "DevOps na Prática"
#
#  test ──► build-and-push ──► deploy (Kind + Helm)
#   │            │                  │
#   │            │                  └─ CD: publica a nova versão no Kubernetes e expõe
via NodePort
#   │            └─ build da imagem Docker e envio ao GitHub Container Registry
(ghcr.io)
#   └─ testes automatizados (Jest + Supertest) com cobertura mínima de 80%

name: CI-CD

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: devops-tarefas-api
  RELEASE: tarefas
  NAMESPACE: devops

jobs:
  # ------------------------------------------------------------------ CI: testes
  test:
    name: 1. Testes automatizados
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: app
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Configurar Node.js 20
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
          cache-dependency-path: app/package-lock.json

      - name: Instalar dependências
        run: npm ci

      - name: Executar testes (Jest)
        run: npm test

      - name: Publicar relatório de cobertura
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: cobertura-testes
          path: app/coverage/
          retention-days: 7

  # ---------------------------------------- CI: build Docker + push registry
  build-and-push:
    name: 2. Build Docker e push (GHCR)
    needs: test
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
```

```
    outputs:
      image: ${{ steps.vars.outputs.image }}
      tag: ${{ steps.vars.outputs.tag }}
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Definir nome e tag da imagem
        id: vars
        run: |
          OWNER="${GITHUB_REPOSITORY_OWNER,,}"   # o registry exige letras minúsculas
          echo "image=${REGISTRY}/${OWNER}/${IMAGE_NAME}" >> "$GITHUB_OUTPUT"
          echo "tag=sha-${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"

      - name: Configurar Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Login no GitHub Container Registry
        if: github.event_name != 'pull_request'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Gerar tags e labels da imagem
        id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ steps.vars.outputs.image }}
          tags: |
            type=raw,value=${{ steps.vars.outputs.tag }}
            type=raw,value=latest,enable={{is_default_branch}}

      - name: Build e push da imagem
        uses: docker/build-push-action@v6
        with:
          context: ./app
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - name: Resumo
        run: |
          echo "### Imagem Docker" >> "$GITHUB_STEP_SUMMARY"
          echo '```' >> "$GITHUB_STEP_SUMMARY"
          echo "${{ steps.meta.outputs.tags }}" >> "$GITHUB_STEP_SUMMARY"
          echo '```' >> "$GITHUB_STEP_SUMMARY"

  # ------------------------------------- CD: deploy no Kubernetes com Helm
  deploy:
    name: 3. Deploy no Kubernetes (Helm)
    needs: build-and-push
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: read
    env:
      IMAGE: ${{ needs.build-and-push.outputs.image }}
      TAG: ${{ needs.build-and-push.outputs.tag }}
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Criar cluster Kubernetes (Kind)
        uses: helm/kind-action@v1.12.0
        with:
          cluster_name: devops
          config: k8s/kind-config.yaml
```

```
      - name: Configurar Helm
        uses: azure/setup-helm@v4

      - name: Preparar namespace e credencial do registry
        run: |
          kubectl create namespace "$NAMESPACE" --dry-run=client -o yaml | kubectl
apply -f -
          kubectl -n "$NAMESPACE" create secret docker-registry ghcr-cred \
            --docker-server="$REGISTRY" \
            --docker-username="${{ github.actor }}" \
            --docker-password="${{ secrets.GITHUB_TOKEN }}" \
            --dry-run=client -o yaml | kubectl apply -f -

      - name: Validar chart Helm
        run: helm lint helm/devops-app

      - name: Deploy / upgrade com Helm
        run: |
          helm upgrade --install "$RELEASE" helm/devops-app \
            --namespace "$NAMESPACE" \
            --set image.repository="$IMAGE" \
            --set image.tag="$TAG" \
            --set "imagePullSecrets[0].name=ghcr-cred" \
            --wait --timeout 5m --atomic

      - name: Verificar rollout e recursos
        run: |
          kubectl -n "$NAMESPACE" rollout status deployment/"$RELEASE"-devops-app --
timeout=120s
          kubectl -n "$NAMESPACE" get deploy,rs,pods,svc -o wide
          helm list -n "$NAMESPACE"

      - name: Teste de fumaça via NodePort (30080)
        run: |
          for i in $(seq 1 10); do
            curl -fsS http://localhost:30080/health && break || sleep 3
          done
          echo
          curl -fsS http://localhost:30080/api/info | tee info.json
          echo
          grep -q "\"versao\":\"$TAG\"" info.json && echo "Versão $TAG publicada com
sucesso no cluster"

      - name: Helm test
        run: helm test "$RELEASE" -n "$NAMESPACE" --logs

      - name: Resumo do deploy
        if: always()
        run: |
          {
            echo "### Deploy no Kubernetes"
            echo "Release **$RELEASE** - imagem \`$IMAGE:$TAG\`"
            echo '```'
            kubectl -n "$NAMESPACE" get pods,svc -o wide
            echo '```'
          } >> "$GITHUB_STEP_SUMMARY"

  # ------------- CD opcional: deploy no Minikube local via runner self-hosted
  # Ative criando a variável de repositório DEPLOY_MINIKUBE=true e registrando
  # um runner self-hosted (Settings > Actions > Runners) na máquina do Minikube.
  deploy-minikube:
    name: 4. Deploy no Minikube local (opcional)
    needs: build-and-push
    if: github.event_name != 'pull_request' && vars.DEPLOY_MINIKUBE == 'true'
    runs-on: self-hosted
    permissions:
      contents: read
      packages: read
    env:
      IMAGE: ${{ needs.build-and-push.outputs.image }}
      TAG: ${{ needs.build-and-push.outputs.tag }}
```

```
    steps:
      - name: Checkout do código
        uses: actions/checkout@v4

      - name: Credencial do registry
        run: |
          kubectl create namespace "$NAMESPACE" --dry-run=client -o yaml | kubectl
apply -f -
          kubectl -n "$NAMESPACE" create secret docker-registry ghcr-cred \
            --docker-server="$REGISTRY" \
            --docker-username="${{ github.actor }}" \
            --docker-password="${{ secrets.GITHUB_TOKEN }}" \
            --dry-run=client -o yaml | kubectl apply -f -

      - name: Deploy / upgrade com Helm
        run: |
          helm upgrade --install "$RELEASE" helm/devops-app \
            --namespace "$NAMESPACE" \
            --set image.repository="$IMAGE" \
            --set image.tag="$TAG" \
            --set "imagePullSecrets[0].name=ghcr-cred" \
            --wait --timeout 5m --atomic
          kubectl -n "$NAMESPACE" get pods,svc -o wide
```

APÊNDICE B – DOCKERFILE E CHART HELM

Dockerfile da aplicação (app/Dockerfile):

```
# ---------- Etapa 1: dependências de produção ----------
FROM node:20.18.1-alpine3.20 AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

# ---------- Etapa 2: imagem final enxuta ----------
FROM node:20.18.1-alpine3.20
LABEL org.opencontainers.image.title="devops-tarefas-api" \
      org.opencontainers.image.description="API de tarefas - Atividade DevOps na
Prática"
ENV NODE_ENV=production \
    PORT=3000
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY package.json ./
COPY src ./src
COPY public ./public
# Diretório de logs gravável pelo usuário sem privilégios (lido pelo Filebeat)
RUN mkdir -p /var/log/app && chown -R node:node /var/log/app
USER node
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD wget -qO- http://127.0.0.1:3000/health || exit 1
CMD ["node", "src/server.js"]
```

Valores do chart (helm/devops-app/values.yaml):

```
# Valores padrão do chart. O pipeline (GitHub Actions) sobrescreve
# image.repository e image.tag a cada execução com --set.

replicaCount: 2

image:
  repository: ghcr.io/SEU-USUARIO/devops-tarefas-api
  tag: latest
  pullPolicy: IfNotPresent

# Segredo para baixar imagens privadas do GHCR (criado pelo pipeline)
imagePullSecrets: []
#  - name: ghcr-cred

nameOverride: ""
fullnameOverride: ""

app:
  port: 3000
  environment: kubernetes
  # Grava também em arquivo para coleta por um Filebeat (opcional)
  logFile: ""

service:
  # NodePort expõe a aplicação fora do cluster (exigência da Etapa 3)
  type: NodePort
  port: 80
  nodePort: 30080

ingress:
  enabled: false
  className: nginx
  host: tarefas.local

resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
```

```
    cpu: 250m
    memory: 192Mi

autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 5
  targetCPUUtilizationPercentage: 70

podSecurityContext:
  runAsNonRoot: true
  runAsUser: 1000
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities:
    drop: ["ALL"]
```

Deployment (helm/devops-app/templates/deployment.yaml):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "devops-app.fullname" . }}
  labels:
    {{- include "devops-app.labels" . | nindent 4 }}
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ .Values.replicaCount }}
  {{- end }}
  # Rolling update sem indisponibilidade: sobe um pod novo antes de remover o antigo
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      {{- include "devops-app.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "devops-app.selectorLabels" . | nindent 8 }}
      annotations:
        # Força novo rollout quando a configuração muda
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
sha256sum }}
    spec:
      {{- with .Values.imagePullSecrets }}
      imagePullSecrets:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      securityContext:
        {{- toYaml .Values.podSecurityContext | nindent 8 }}
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag |
default .Chart.AppVersion }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          securityContext:
            {{- toYaml .Values.securityContext | nindent 12 }}
          envFrom:
            - configMapRef:
                name: {{ include "devops-app.fullname" . }}
          ports:
            - name: http
              containerPort: {{ .Values.app.port }}
              protocol: TCP
          readinessProbe:
            httpGet:
              path: /health
              port: http
            initialDelaySeconds: 3
```

```
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /health
              port: http
            initialDelaySeconds: 10
            periodSeconds: 15
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
          volumeMounts:
            - name: logs
              mountPath: /var/log/app
            - name: tmp
              mountPath: /tmp
      volumes:
        - name: logs
          emptyDir: {}
        - name: tmp
          emptyDir: {}
```

Service (helm/devops-app/templates/service.yaml):

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "devops-app.fullname" . }}
  labels:
    {{- include "devops-app.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type }}
  selector:
    {{- include "devops-app.selectorLabels" . | nindent 4 }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: http
      protocol: TCP
      {{- if and (eq .Values.service.type "NodePort") .Values.service.nodePort }}
      nodePort: {{ .Values.service.nodePort }}
      {{- end }}
```

APÊNDICE C – CONFIGURAÇÃO DA ELASTIC STACK

Configuração do Filebeat (monitoring/filebeat/filebeat.yml):

```
# Filebeat – coleta os logs JSON da API de Tarefas e envia ao Elasticsearch
filebeat.inputs:
  - type: filestream
    id: tarefas-api-logs
    paths:
      - /var/log/app/*.log
    parsers:
      # Cada linha é um objeto JSON (NDJSON); os campos vão para a raiz do evento
      - ndjson:
          target: ""
          overwrite_keys: true
          add_error_key: true
    fields:
      origem: tarefas-api
    fields_under_root: true

processors:
  - add_host_metadata: ~
  - drop_fields:
      fields: ["agent.ephemeral_id", "ecs.version"]
      ignore_missing: true

output.elasticsearch:
  hosts: ["http://elasticsearch:9200"]

# Garante o tipo numérico decimal para o tempo de resposta (campo próprio da app)
setup.template.append_fields:
  - name: response_time_ms
    type: float
  - name: origem
    type: keyword

setup.kibana:
  host: "http://kibana:5601"

logging.level: info
```

Orquestração dos containers (monitoring/docker-compose.yml):

```
# Etapa 4 – Logging e Métricas com Elastic Stack (Filebeat + Elasticsearch + Kibana)
#
#   app ──(grava JSON em /var/log/app/app.log)──▶ volume app-logs
#   filebeat ──(lê o volume, decodifica o JSON)──▶ elasticsearch ──▶ kibana
#   trafego  ──(gera requisições contínuas na app para alimentar o dashboard)
#
# Uso:  docker compose up -d   →  Kibana em http://localhost:5601
name: devops-monitoring

services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.17.0
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false        # apenas para ambiente acadêmico/local
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
      - cluster.routing.allocation.disk.threshold_enabled=false
    ports:
      - "9200:9200"
    volumes:
      - es-data:/usr/share/elasticsearch/data
    healthcheck:
      test: ["CMD-SHELL", "curl -fs http://localhost:9200/_cluster/health | grep -qE
'\"status\":\"(green|yellow)\"'"]
      interval: 10s
      timeout: 5s
      retries: 30
```

```
  kibana:
    image: docker.elastic.co/kibana/kibana:8.17.0
    container_name: kibana
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
    ports:
      - "5601:5601"
    depends_on:
      elasticsearch:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "curl -fs http://localhost:5601/api/status | grep -q
'\"level\":\"available\"'"]
      interval: 10s
      timeout: 5s
      retries: 40

  app:
    build: ../app
    image: devops-tarefas-api:local
    pull_policy: build
    container_name: app
    environment:
      - APP_ENV=docker-compose
      - LOG_FILE=/var/log/app/app.log
    ports:
      - "3000:3000"
    volumes:
      - app-logs:/var/log/app

  filebeat:
    image: docker.elastic.co/beats/filebeat:8.17.0
    container_name: filebeat
    user: root
    command: ["filebeat", "-e", "--strict.perms=false"]
    volumes:
      - ./filebeat/filebeat.yml:/usr/share/filebeat/filebeat.yml:ro
      - app-logs:/var/log/app:ro
      - filebeat-data:/usr/share/filebeat/data
    depends_on:
      elasticsearch:
        condition: service_healthy
      app:
        condition: service_started

  trafego:
    image: curlimages/curl:8.11.1
    container_name: trafego
    entrypoint: ["/bin/sh", "/gerar-trafego.sh"]
    volumes:
      - ./gerar-trafego.sh:/gerar-trafego.sh:ro
    depends_on:
      - app

volumes:
  es-data:
  app-logs:
  filebeat-data:
```

Comandos para reproduzir o ambiente:

```
# Etapa 4 - Elastic Stack
cd monitoring
docker compose up -d --build
./kibana/importar-dashboard.sh
# Kibana: http://localhost:5601/app/dashboards#/view/tarefas-dashboard

# Etapa 3 - deploy manual no Minikube
minikube start
docker build -t devops-tarefas-api:local app
minikube image load devops-tarefas-api:local
helm upgrade --install tarefas helm/devops-app -n devops --create-namespace \
```

```
  --set image.repository=devops-tarefas-api --set image.tag=local \
  --set image.pullPolicy=Never
minikube service tarefas-devops-app -n devops --url
helm test tarefas -n devops --logs
```

APÊNDICE D – ROTEIRO DA APRESENTAÇÃO ORAL (ATÉ 10 MINUTOS)

Quadro 9 – Roteiro da apresentação

Tempo	Conteúdo	Demonstração
0:00–1:00	Apresentação do grupo, objetivo e arquitetura (Figura 1)	Slide com o diagrama
1:00–2:30	Conceitos: DevOps × Ágil; CI × CD (Delivery) × CD (Deployment); VMs × containers	Analogias do restaurante, da fábrica de bolos e do prédio
2:30–3:30	Aplicação e testes automatizados	npm test com cobertura
3:30–5:30	Pipeline no GitHub Actions: testes, build, push e deploy	Fazer um commit ao vivo (ex.: mudar o subtítulo da página) e acompanhar o pipeline; mostrar a imagem no GHCR
5:30–7:00	Kubernetes e Helm: chart, NodePort, rolling update, rollback	kubectl get pods,svc; abrir a aplicação no NodePort; helm test
7:00–9:00	Monitoramento: Filebeat → Elasticsearch → Kibana	Abrir o dashboard; chamar /api/erro e /api/lento e ver os gráficos mudarem
9:00–10:00	Desafios e reflexão crítica; encerramento	Quadro de desafios

Fonte: Elaborado pelos autores (2026).